

RIPAR WALLET

v1 build guide · an air-gapped, pulse-verified hardware signer



Screen side: screen, thumb pulse pad, SIGN pin and USB-C ports



Lens side: the camera reads QR codes through the lens bezel

Contents

1. What it is, and the parts you need
2. Printing it on the Bambu Lab P1S
3. Wiring every module
4. Putting it together
5. Testing and charging
6. Firmware
7. Hackathon quick sheet (Monad Metropolis)
8. Where everything lives in the repo

What it is, and the parts you need

Ripar is a pocket-camera-shaped signer that **never touches the internet**. The keys stay inside it and it has no Wi-Fi or Bluetooth. Everything goes in and out as QR codes:

1. The companion app shows a request as a QR code (animated if it's long).
2. Ripar's camera scans it, and Ripar decodes it **itself** to show what you are signing: the amount, the payee, the contract and the risk notes.
3. You rest your thumb on the pulse pad. The MAX30102 has to see a **live heartbeat**.
4. You press **SIGN**, the pin on the left end.
5. Ripar shows the signature as a QR code. The companion scans it and sends it to Monad.

Parts list

Part	Job	Notes
Waveshare ESP32-S3-LCD-2	Brain + 2-inch screen	ESP32-S3R8, 8 MB PSRAM, 16 MB flash, 240×320 display (used landscape)
OV5640 camera	QR scanner	Already plugged into the board's camera connector
MAX30102	Pulse sensor under the thumb	I2C, 4 wires
18650 Li-ion cell, 3.7 V 2000 mAh	Battery	Single cell only; up to 19.3 mm thick with tape and wires
TP4056 USB-C charger (with protection)	Charge-only port	Pads B+ B- OUT+ OUT-
12×12 mm latching push switch + blue cap	Power ON/OFF	Goes in from outside, through the square hole on top
12 mm buzzer	Beeps (frame read, pulse OK, signed)	GPIO18
INMP441 microphone	Later (camera version)	Mounted in the case but not wired for the wallet
Printed parts	Case	Front shell, back shell, 3× SIGN pin, 2 fit-test coupons (all on one plate)
Consumables		Female-to-female jumper wires, solder, heat-shrink or electrical tape

The enclosure at a glance

Item	Value
Size	100 × 62 × 32 mm, landscape and camera-shaped
Closing	No screws. A 1.26 mm lip on the front shell clicks into a groove in the back shell (0.25 mm snap bead). A fingernail notch on the bottom seam opens it.
Board mounting	Screen glass against the window. The top and left walls plus two fences locate the board, and four flat pegs press on its M2 nuts when you close the case.
Cradles	18650 saddles, TP4056 cradle, switch cradle, pulse-sensor pocket (0.7 mm skin, 10×8 mm thumb window), mic rails, buzzer cradle
Camera	33° lens hood; even the full 68° field of view isn't clipped
Lettering	"RIPAR WALLET" on the top and bottom, "RIPAR" on the lens face
Checks	SolidWorks interference detection found 0 unexpected collisions. All STLs are watertight, and three independent design reviews were applied.

Printing it on the Bambu Lab P1S

The file is ready: `print/bambu/RiparWallet_ALL_P1S.gcode.3mf` . It has every part on one plate and is already sliced: **about 2 h 23 min, 69.5 g of PLA**. Open it in Bambu Studio and press **Print plate**, or copy it to the printer's microSD card.

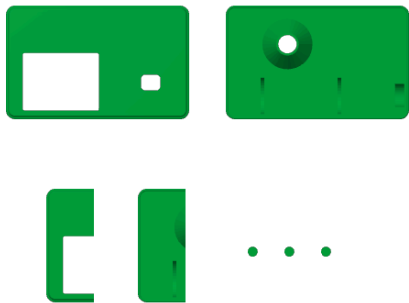
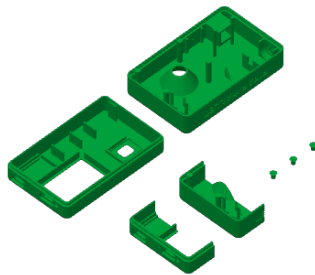


Plate layout from above: shells at the back, fit-test coupons and 3 SIGN pins at the front



Bambu Studio's plate preview

Settings baked into the file

Setting	Value	Why
Printer / nozzle	Bambu Lab P1S, 0.4 mm	
Filament	Bambu PLA Basic (PETG also fits)	Use black or dark for both shells. A dark case hides light leaks around the pulse sensor, and a light-coloured lens hood flares the camera.
Layer height	0.16 mm	Crisp lettering and an accurate snap bead
Walls / infill	3 walls, 15 % gyroid	Stiff walls for the snap-fit
Plate	Textured PEI	Change it in Bambu Studio and re-slice if you use another plate
Supports	Off	Every part was designed to print without them
Brim	Auto	Only the tiny SIGN pins get one
Orientation	Front shell screen-face down, back shell lens-face down	Already set in the file

The two fit-test coupons

These are the left 26 mm of each shell: the end with the USB-C ports, SIGN pin, board corner and snap lip. They let you check the fit with little plastic.

- **Printing everything in one go?** Delete the two coupons in Bambu Studio and press **Slice plate**. That saves about 30 min and 16 g.

- **Want to be safe?** Print only the coupons first (about 40 min). Check that the board drops in, a USB-C cable clicks home, the SIGN pin clicks the BOOT key and the two halves snap. Then print the shells.

Measure your parts before closing the case

Part	Designed for	Setting in <code>cad/design.py</code>
Big latching switch	12×12 mm body, housing ≤ 12 mm tall, cap ≤ Ø12 mm	<code>SW</code> , <code>SW_CRADLE_CLR</code>
MAX30102 board	Up to 20.3 × 15.3 mm, header soldered on the back	<code>MAX</code>
18650	Up to 19.3 mm thick with tape, ≤ 69.5 mm long with tabs	<code>BAT_D</code> , <code>BAT_L</code>
TP4056	28 × 17.3 mm, 1.0–1.6 mm PCB	<code>TP</code>
USB-C cables	Plug overmold ≤ about 12.6 × 6.8 mm	<code>USB_RECESS</code>

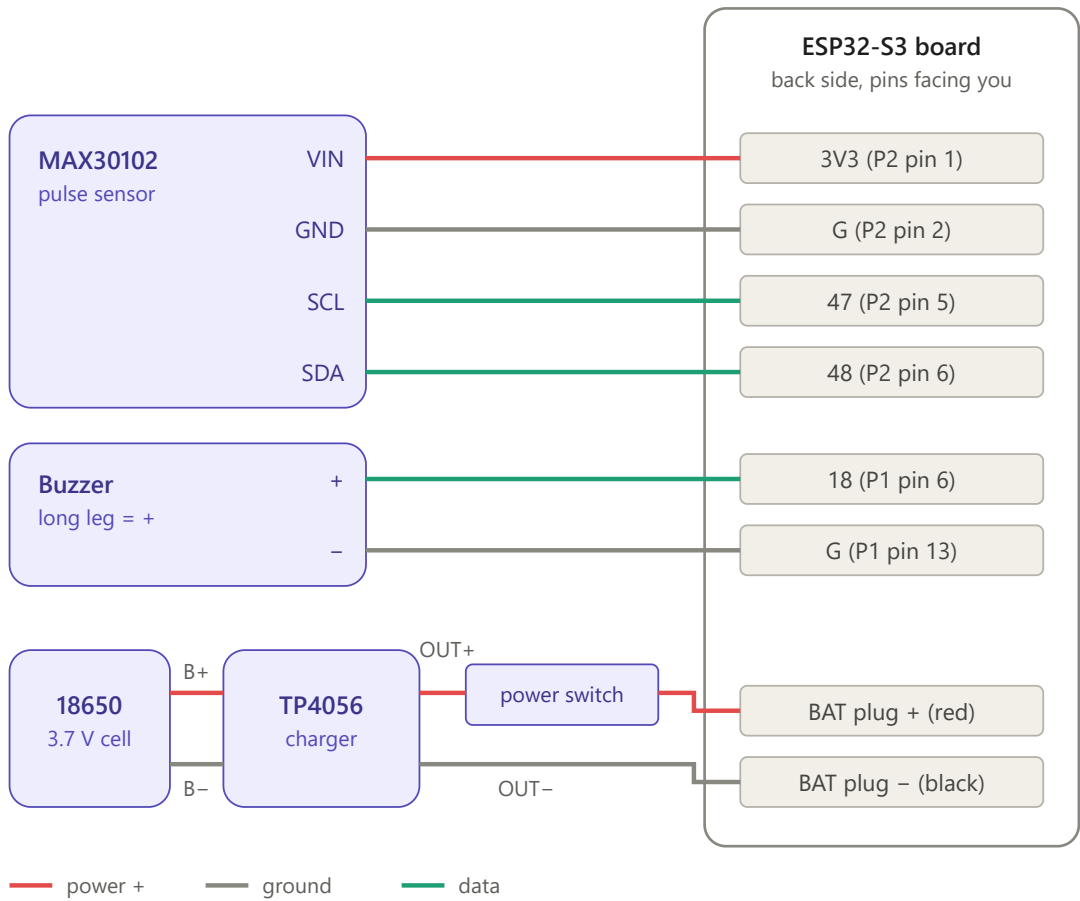
If something is off

Symptom	Fix in <code>cad/design.py</code> , then run <code>python cad/make_all.py</code>
Lid won't stay closed	<code>BEAD</code> +0.1
Lid too hard to close	<code>BEAD</code> -0.1 or <code>LIP_CLR</code> +0.05
Seam stays open by a hair with the board fitted	<code>PEG_GAP</code> 0.1 → 0.2
Board loose in the window	<code>FENCE_CLR</code> 0.2 → 0.1
USB-C cable doesn't seat	<code>USB_RECESS</code> depth 2.0 → 2.2
Switch tight or rubbing	<code>SW_CRADLE_CLR</code> +0.1

The rebuild takes about 15–20 min. It regenerates the SolidWorks parts, runs the interference check and re-slices for the P1S automatically.

Wiring every module

You need only **4 wires for the pulse sensor, 2 for the buzzer, and a short power chain**. The camera is already plugged in. The SIGN key needs no wire, because the printed pin presses the board's BOOT button.



Everything you need to wire for the wallet build

Finding the pins

Turn the board over so the header pins point at you, with the USB-C at the **bottom**. Every pin is labelled on the board.

#	Left row (P1), top to bottom	Right row (P2), top to bottom
1–7	2, 4, 6, 16, 17, 18 , 21	3V3 , G , TX (43), RX (44), 47 , 48 , 15
8–14	8, 7, 10, 20, 19, G , 5V	13, 11, 12, 14, 9, G, VBAT

With the camera plugged in, it uses GPIO 2, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17 and 21. GPIO 19 and 20 are USB. The only free pins are **18, 43 and 44**, plus the I2C pair **47/48**, which the pulse sensor shares with the board's motion sensor.

Connections, one module at a time

1. Pulse sensor (MAX30102): 4 wires

Sensor pin	Board pin	Wire
VIN	3V3 (right row, 1st)	red
GND	G (right row, 2nd)	black
SCL	47 (right row, 5th)	any colour
SDA	48 (right row, 6th)	any colour

- Leave INT (and IRD/RD if your board has them) unconnected.
- Solder the sensor's header on the **back** of its board, so the sensor face sits flat against the thumb window.
- On the purple board, move the small "1.8V / 3.3V" solder jumper to **3.3V**. Otherwise the ESP32 can't read it reliably.

2. Buzzer: 2 wires

Buzzer	Board pin
Long leg (+)	18 (left row, 6th)
Short leg (–)	G (left row, 13th)

A small 3.3 V active buzzer can go straight onto the pin. A 5 V or magnetic buzzer needs an S8050 transistor with a 1 kΩ base resistor and a 1N4148 across the buzzer. If yours is a 3-pin module (VCC/GND/I/O), it needs a split 3V3 wire.

3. Power chain: do this last, with the switch OFF

1. **Battery + (red) → TP4056 "B+"** and **battery – (black) → TP4056 "B–"**. Don't use the TP4056's IN+/IN– pads; those are its USB input.
2. **TP4056 "OUT+" → power switch → red wire of the board's BAT plug**. The switch has 6 legs in 2 rows of 3. Use the **middle leg (COM)** and **one end leg of the same row (NO)**. With a multimeter on beep mode, clicking ON beeps and clicking OFF stops.
3. **TP4056 "OUT–" → black wire of the BAT plug**.
4. Plug the BAT cable into the small white connector on the back of the board. **Red goes to ⊕**; the board marks ⊖ and ⊕ beside it.
5. **No BAT cable?** Use the **VBAT** header pin instead (pin 14 of the right-hand column, next to 5V): the switch goes to VBAT, and OUT– goes to any G pin. First check with a multimeter's beep mode that VBAT connects to the + pin of the white socket. **Never put the battery on 5V or 3V3**. For bench work, the board's USB-C alone powers everything.

4. Nothing to wire

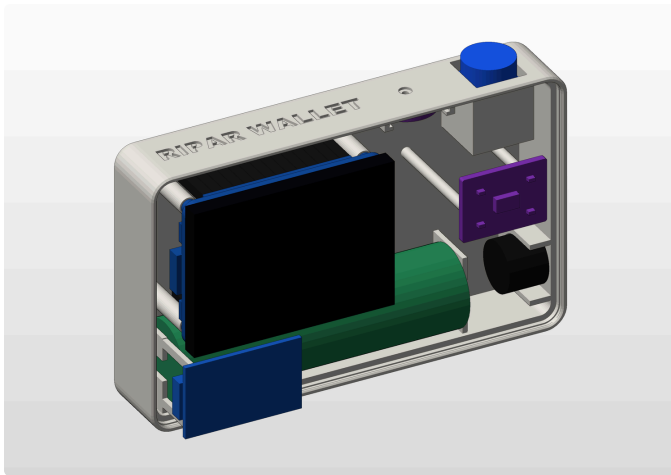
- **Camera:** already in its connector. Tape it flat on the back of the board, over the TF slot, which is where the lens hood expects it.
- **SIGN key:** the printed pin presses the board's BOOT button (GPIO0).

- **INMP441 microphone:** skip it for the wallet, because there are no free pins left for both it and the buzzer. It's mounted for the camera version: SCK→TX(43), WS→RX(44), SD→18 in place of the buzzer.

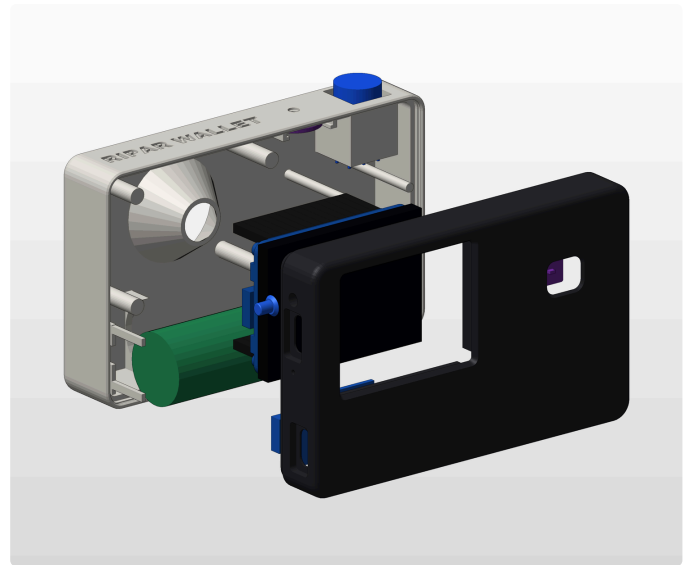
Safety checklist

- Check the battery polarity **twice** before connecting it. Reversed polarity kills the board instantly.
- Wrap every bare wire end and solder joint in tape or heat-shrink. Nothing metal should touch the battery's ends.
- Keep all wires away from the case's rim, so closing it can't pinch them.
- **Never plug both USB-C ports at once.**
- Use a cell with welded tabs. Don't solder straight onto the can.

Putting it together



Inside, with the front shell removed



Exploded view: everything drops into a printed cradle

Front shell (screen face down on the table)

1. Drop the **SIGN pin** into its hole from the inside, flange inside.
2. Lay the **ESP32 board** glass-down in the window pocket. The USB-C goes to the **left**, and the board edges sit against the top and left walls, inside the right-hand and bottom fences.
3. Lay the **TP4056** in its cradle at the bottom-left: bare side toward the screen, USB-C into the **lower left** opening.
4. Press the **MAX30102** into the thumb-pad pocket, sensor toward the window, wires out of the open right side.

Back shell (lens face down)

1. Lay the **18650** in the saddles, wires out through the slots at each end.
2. The **big switch goes in from outside**. Solder two wires (COM and NO), feed them in through the square hole in the top face and out through the cradle floor, then push the switch in until it sits on the floor. Press the blue cap on last. If it's loose, wrap one layer of Kapton tape around it.
3. Slide the **INMP441** into the rails under the top wall, tiny-hole side against the wall.
4. Drop the **buzzer** into the U-cradle at the right end, sound hole toward the grille.

Close and open

1. Plug in all the jumper wires (chapter 3) and tuck them away from the rim.
2. Line up the shells and press all the way round until the bead clicks. The four pegs land on the board's M2 nuts and hold the screen against the window.
3. **To open:** put a fingernail or coin in the notch on the bottom seam (right side) and twist.

Testing and charging

Test	How	Expected
1. Board only	Plug the board's USB-C (upper port on the left end) into your PC	The screen lights up once the firmware is flashed (chapter 6)
2. Battery	Unplug USB and click the big button ON	The board runs from the battery; click again to turn it off
3. Charging	USB-C into the lower port (the TP4056)	Red LED while charging, blue or green when full. It charges even when switched off.
4. Pulse	Rest your thumb on the pad	The heart animation beats and bpm settles within about 8 s
5. SIGN pin	On Home, press the pin on the left end	Click; the camera viewfinder opens. Hold 2 s to cancel.
6. Camera	Start a scan and point the lens at a QR code	The viewfinder shows the code and a progress bar

Never plug both USB-C ports at the same time. The board's own port powers it even when the switch is off.

Charging speed. The TP4056 charges at 1 A by default and gets warm inside a PLA box. Either print the front shell in PETG, or swap its R_PROG resistor to 2.4 kΩ for 0.5 A.

Firmware

Status: built and tested on the PC. The full ESP32-S3 firmware in `firmware/` (v1.2) builds cleanly: RAM 12.1 %, flash 11.3 %. It passes **10 host test suites**, checked against an independent Python reference, and the browser emulator runs the same code (570 checks). v1.2 derives the vault from the device's own key and has the contract addresses built in. There is **no Wi-Fi or Bluetooth code** in the binary. **It has not run on a real board yet**, so the first flash is also the first hardware test.

What runs on the device

- **Two keys, generated on the device.** The seed comes from hardware randomness, raw pulse-sensor samples and camera noise. **K1** (secp256k1) owns the vault and signs only mandates. **P1** (P-256) signs co-signatures, deny, PANIC, revoke, reopen and Privy requests; Monad checks it natively.
- **No radio.** Data moves only as QR codes in the BC-UR format; long requests arrive as animated QR codes.
- **Signs only what it shows.** Every signature is rebuilt from the lines on screen. SIGN only works after the last line has been shown, and only while the pulse still counts as live: at least 5 beats in 8 s, 40–180 bpm.
- **Refuses with the exact reason.** Examples: MANDATE WITHOUT PULSE CO-SIGN, a contract or chain that isn't pinned, UNKNOWN CALldata, a token it can't show safely, an expiry too far ahead, and any Privy change outside the allowlist.

First boot

1. The RIPAR logo appears. The device then tests its crypto against published test vectors and cross-checks it with mbedTLS. If a check fails, it shows **SELFTEST FAIL** and never signs anything.
2. On the first run only, it shows "NEW DEVICE: creating keys".
3. Home shows the short K1 address, the battery level and **NOT PAIRED**.

The SIGN key (BOOT, GPIO0)

The key has three actions: **press** (let go within 1 s), **hold 2 s** and **hold 5 s**. A press or hold only works on the screen where it started, so a hold that cancels a screen can never become a PANIC or a deny on the next one.

Screen	Press	Hold 2 s	Hold 5 s
Home	Scan a request	Open the Hold screen	(only from the Hold screen)
Hold		Let go before 5 s to show the pairing QR	PANIC : signed at once, no pulse, because it can only tighten
Scan		Cancel	
Review	Next page. On the last page: continue to Pulse (or sign a deny)	Co-sign review: DENY . Other reviews: cancel	
Pulse	Ignored	Cancel	
Armed ("PULSE OK - press SIGN")	SIGN	Cancel	
Signed QR / message	Home	Home	
Pairing QR	Home	Device menu	
Device menu	Next item	Select: REVOKE last mandate / REOPEN agent lane / BACK	

The review shows up to 9 rows at a time and each press moves 8 rows, so every row is seen. Every screen goes back to Home after 120 s with no key press. The buzzer ticks for each QR part and each heartbeat. A rising tone means OK; a falling tone means refused.

What it can sign

Request (QR in)	Device shows	Reply (QR out)
<code>ripar-pair-req</code>	The chain and every contract to pin, with a red CHANGES line for anything different	<code>ripar-pair</code> : both keys bound together
<code>ripar-mandate-req</code>	Every rule in plain language. It refuses unknown rules and mandates without the pulse co-sign.	<code>eth-signature</code> from K1
<code>ripar-cosign-req</code>	Amount and token, full payee with a 4-icon fingerprint and NEW PAYEE, expiry (UTC), budget, risk and AI notes	<code>ripar-cosign</code> : P1 signature plus pulse evidence
<code>ripar-deny-req</code>	The agent and the request	<code>ripar-deny</code>
<code>ripar-privy-req</code>	The allowed Privy change, shown in full	<code>ripar-der-sig</code>

Flashing and first try

Flash over USB-C

1. Plug the **board's** USB-C (the upper port on the left end) into the PC. Don't use the TP4056 port, and never plug both at once.
2. In the repo's `firmware` folder, run `pio run -e ripar -t upload`.
3. Optional: `pio device monitor -b 115200` shows the self-test and the K1 address.
4. **If the upload can't connect:** hold BOOT (the SIGN pin), tap RST (the pinhole), release BOOT, upload again, then tap RST.
5. Re-flashing keeps the keys and the pairing. `pio run -t erase` **wipes the seed**, which means a new vault owner, and there is no seed backup yet. Only erase a unit that holds no funds.

Try the whole flow before the contracts exist

These tools run on the online PC. It is untrusted, which is why the device checks every signed reply itself.

- `firmware/tools/make_request.py` builds requests and verifies the device's signed replies.
 - `firmware/tools/companion_lite.html` plays requests as QR codes.
1. Build a request with `python tools/make_request.py build pair chainId=10143 ... docs/FIRMWARE.md` section 6 gives every command.
 2. Open `companion_lite.html`, paste the request and press **Show QR**.
 3. On the device, press SIGN on Home and point the camera at the screen. Read every line, rest your thumb on the pad, then press SIGN.
 4. Scan the device's reply with any QR reader (a phone camera works). Then run the verify command the page prints; exit code 0 means VERIFIED.
 5. No board yet? `python tools/make_request.py simulate @req.txt` plays a demo device, so you can test your requests and verify commands.

Known limitations

- **Not tested on hardware yet.** On your unit, check the screen rotation (`RIPAR_LCD_ROTATION`), the camera orientation and exposure, and the pulse thresholds.
- **Pulse thresholds.** The anti-spoof pulse checks were tuned on simulated signals. Test with your real thumb: at rest it should pass within about 15 s.
- **Prototype security.** The seed is stored in flash without encryption and there is no seed backup. The crypto isn't constant-time.

The full guide is `docs/FIRMWARE.md`, and the byte-level message format is `docs/PROTOCOL.md`.

The whole system: contracts, agent, companion

- **Contracts on Monad.** The **PulseCosignEnforcer** is a rule attached to the agent's MetaMask delegation. The agent may pay small amounts by itself (the AUTO path), only to payees you approved under this mandate, and only while the sentinel lane is open. Anything else needs your device's co-signature. Monad checks it with its P-256 precompile.
- **The agent** (Qwen) pays invoices. When a payment needs a human, it sends an escalation to the companion.
- **The companion** (web app) shows the request as a QR. You check it on the device, thumb on, SIGN. The companion then passes the signature back to the agent.
- **The kill switch.** PANIC (hold 5 s on Home) kills every mandate. Revoke kills one. Chainlink CRE can close the AUTO lane, and only your device can reopen it.

Try it now, with no hardware and no public chain

1. On the PC: `npm install`, then `bash scripts/dev-stack.sh`. This starts a local fork of Monad testnet with the contracts, the agent and the companion.
2. Open `http://127.0.0.1:5173/?devstack` and choose **Device: EMULATOR**. It runs the firmware's own code in the browser, labelled **EMULATOR - DEMO KEYS**.
3. Follow the pages: **Pair**, then **Vault** (deploy it and get MockUSD), then **Mandate**, then **Inbox** ("Ask the agent to run now"). Answer each escalation on the emulated device. The line *Next on the device* tells you which key to press.
4. The prompt-injection invoice is the one to watch. The device shows **AI claims: MISMATCH**. Hold SIGN for 2 s to deny it.

Contract addresses (built into firmware v1.2)

Contract	Address
PulseCosignEnforcer	0x64d61fe5438981DC803ED61250FEf024617ae7eE
RiparDeviceRegistry	0xA08a47c9d645926615CF04D69b7a048133F68c9f
RiparReputationRelay (10143)	0xE433dCA75CA6cd730b1006F51A26208B000eA9E2
MockUSD (10143)	0xB5b7eaffbF9bf68cbcC1Ce8B5850b2ea9d6f9a2a

To deploy them on Monad testnet with your own key, follow `docs/DEPLOY.md`. The addresses are deterministic (CREATE2), so the deployed contracts land exactly where the firmware expects them. The contracts passed an adversarial review (25 findings, 12 confirmed and fixed) and 715 tests.

Hackathon quick sheet: Monad Metropolis

Item	Detail
Deadline	Wed 14 Oct 2026, 03:59 UTC (Tue 13 Oct, 23:59 ET; Wed 14 Oct, 09:29 IST). Aim to submit a complete draft on 12 Oct.
Track	04: Trust, Identity & AI Infrastructure
Pitch	A camera-shaped, air-gapped signer that puts a live human thumb between AI agents and your money. Agents spend inside a mandate; anything more needs your heartbeat, checked on Monad.
On-chain piece	A <code>PulseCosignEnforcer</code> caveat for MetaMask delegations. Small, pre-approved spends go through automatically. New payees, larger amounts, or anything after the risk sentinel trips need Ripar's P-256 co-signature, which Monad verifies natively.
Sponsor integrations planned	Qwen (the treasury agent), Privy (server wallet owned by the device key), Nansen (payee risk), Mera (encrypted agent memory), Chainlink CRE (risk sentinel), Envio (trust dashboard), Alchemy (RPC and webhooks)
Submission needs	A working product, a public project page with a demo video (≤ 3 min), a short write-up and a link to the code (with a LICENSE)

One project per participant. If your team enters more than one project, each one must be submitted by a different team member.

First things to do

1. Register on the hackathon platform if you haven't; confirm the registration close date on the dashboard.
2. Day-1 technical check: **camera QR scanning** on the real board. Fallback: requests from the TF card plus a clearly labelled browser emulator.
3. Print the case, wire it, flash the firmware, and record the demo flow early: pair → agent spends → escalation → pulse + SIGN → on-chain.

Where everything lives in the repo

Path	What
<code>print/bambu/RiparWallet_ALL_P1S.gcode.3mf</code>	Ready-to-print P1S plate (everything on one plate)
<code>print/stl/</code> , <code>print/step/</code>	Print-oriented STLs and STEP files of every part, plus the full assembly
<code>print/PRINTING.md</code>	Print and assembly guide (text version)
<code>docs/WIRING.md</code>	Wiring (text version)
<code>docs/PROTOCOL.md</code>	Device ↔ companion ↔ contract protocol
<code>docs/FIRMWARE.md</code>	Firmware: build, flash, screens and keys, walkthrough, security model
<code>contracts/</code> , <code>docs/DEPLOY.md</code>	Solidity contracts (Foundry), <code>SPEC.md</code> , deploy guide
<code>agent/</code> , <code>companion/</code> , <code>packages/protocol/</code>	AI agent service, companion web app, TypeScript protocol library
<code>firmware/emu/</code> , <code>scripts/dev-stack.sh</code>	Browser emulator of the device; one-command local stack
<code>firmware/</code>	ESP32-S3 firmware (PlatformIO): <code>src/</code> , <code>include/</code> , host tests in <code>test/host/</code> , companion tools in <code>tools/</code>
<code>model/RiparWallet.SLDASM</code> , <code>model/parts/</code>	SolidWorks 2026 assembly and native parts
<code>cad/design.py</code>	Every enclosure dimension in one file
<code>cad/make_all.py</code>	One command to rebuild the parts, assembly, checks, exports, P1S slice and renders
<code>research/</code>	CAD design reviews and researched part dimensions